

Question Bank — Week 2

Complexity and the Array · control flow, functions, errors (Lecture 02, Lab 02) —
Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Questions: *week02-questions.md*. Commit to your own answer before reading one here.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M08	a	M15	d	M22	c
M02	c	M09	b	M16	b	M23	d
M03	a	M10	c	M17	a	M24	a
M04	d	M11	d	M18	c	M25	b
M05	a	M12	a	M19	d	M26	c
M06	c	M13	b	M20	a		
M07	d	M14	c	M21	b		

W2-M01 — b. Keep the fastest-growing term, drop its constant: $3n^2 \rightarrow n^2$. It is also $O(n^3)$, but not $\Theta(n^3)$ (c) — Θ is a tight bound.

W2-M02 — c. $n \leq 1 \cdot n^2$ for all $n \geq 1$. O is an *upper* bound, so a slower-growing function is O of a faster one. (a) and (d) put a faster function under a slower one; (b) claims n grows at least as fast as n^2 .

W2-M03 — a. Constants **you choose**, and only from some n_0 on. (b) has no constant and no n_0 ; (c) is equality, not a bound; (d) is the definition of Ω .

W2-M04 — d. Worst case: the target is last or absent, so every element is compared once.

W2-M05 — a. $(1 + 2 + \dots + n)/n = (n + 1)/2$. Still $O(n)$ — half is a constant.

W2-M06 — c. Halving until 1 takes about $\log_2 n$ steps. $\Theta(n/2)$ (d) would be subtracting, not dividing — and it is just $\Theta(n)$ anyway.

W2-M07 — d. $0 + 1 + \dots + (n - 1) = n(n - 1)/2$. The answer must be a function of n only, so (c) is not an answer.

W2-M08 — a. The inner loop is a constant 5: $5n = \Theta(n)$. Nested loops multiply their *iteration counts*, not their nesting depth.

W2-M09 — b. A list is an array; `in` checks items one by one. Sets and dicts are the hash tables (a).

W2-M10 — c. `append` writes at the end — nothing moves — and the occasional resize averages out (Week 4). The other three shift or search: $O(n)$.

W2-M11 — d. Address arithmetic. It has nothing to do with order (a) or caching (b).

W2-M12 — a. Slots 2, 3, 4, 5 move right: $\text{size} - \text{index} = 6 - 2 = 4$.

W2-M13 — b. Moving right, start at the right, or you overwrite values before you move them (see W2-S2).

W2-M14 — c. The course Array has no negative indices, like C.

W2-M15 — d. `len` is the capacity, fixed at creation. Tracking how many slots are in use is your job.

W2-M16 — b. $r \times \text{cols} + c = 2 \times 4 + 1 = 9$.

W2-M17 — a. Two indices and one temporary for the swap, whatever `n` is.

W2-M18 — c. A structure may not store its data in a Python `list`. (a) and (b) build on the student's own structures — allowed; (d) is a list used as a return value — the interface, allowed.

W2-M19 — d. A block of references plus a size and a capacity; it grows by copying into a bigger block.

W2-M20 — a. Sequence adds: $O(n + n^2) = O(n^2)$.

W2-M21 — b. Each iteration hides **two** $O(n)$ operations: the slice copies up to `n` items and `in` searches them. `n` iterations of $O(n)$ work.

W2-M22 — c. Think of it as “no break”.

W2-M23 — d. The default list is created **once**, when `def` runs, and shared by every call that uses it.

W2-M24 — a. `and` binds tighter: `0` or `('x' and '')`. `'x'` and `''` gives `''` (the last value looked at); `0` or `''` gives `''`.

W2-M25 — b. Start 2, step 3, stop **before** 11: 2, 5, 8.

W2-M26 — c. `return 1` is decided, then `finally` runs (printing `bye`) before the function actually returns; then `print` shows 1.

Part B — Short answer and essay

W2-E1 (4)

- **Lower-order terms** stop mattering as `n` grows: in $\frac{1}{2}n^2 - \frac{1}{2}n$, the n^2 term is 99.9% of the total by `n` = 1,000.

- **Constant factors** are not properties of the algorithm: they change with the machine, the language and how you count steps. What stays the same everywhere is the **shape** of growth.
- **Misleading when n is small**, or constants are huge: a $1000n$ algorithm is worse than an n^2 one for every $n < 1,000$; linear search beats binary search on tiny arrays.
- **Measuring** (viz. `complexity.measure`) shows the constants and the crossover point that Big-O hides; counting shows the shape that one machine's timings cannot prove. You need both.

W2-E2 (4)

- $f = O(g)$: there exist $c > 0, n_0$ with $f(n) \leq c g(n)$ for all $n \geq n_0$ — an upper bound.
- $f = \Omega(g)$: there exist $c > 0, n_0$ with $f(n) \geq c g(n)$ for all $n \geq n_0$ — a lower bound.
- $f = \Theta(g)$: both — $c_1 g(n) \leq f(n) \leq c_2 g(n)$ eventually — a tight bound.
- For $3n^2 + 2n$: it is $O(n^2)$, $\Omega(n^2)$, $\Theta(n^2)$; it is also $\Omega(n)$. **True but uninformative:** $3n^2 + 2n = O(n^3)$ (or $O(2^n)$).

W2-E3 (3)

- **Best:** the target is the first element — 1 comparison, $O(1)$.
- **Worst:** last or absent — n comparisons, $O(n)$.
- **Average** (present, uniformly placed): $(n + 1)/2$, still $O(n)$ — and the average needs a stated assumption about the inputs.
- "Linear search is $O(n)$ " means the **worst case**: it is a guarantee that holds for every input of size n , needs no assumption, and is what you plan capacity around.

W2-E4 (4)

- An array is contiguous, equal-sized slots, so the address of `a[i]` is `base + i × size`: one multiplication and one addition, independent of $n \rightarrow O(1)$.
- Inserting at index i in an array of size n needs every element from i to the end moved one place right: $n - i$ moves, n in the worst case $\rightarrow O(n)$.
- **Fixed size:** the memory after the block may belong to something else, so the array cannot grow in place. When it is full, you must allocate a **new, larger** array and **copy** every element across: $O(n)$ for that operation.
- (Doubling the capacity each time makes appends $O(1)$ amortised — Week 4.)

W2-E5 (4)

- **Rule:** a data structure in `dsa/` stores its data only in the course `Array`, in node objects, or in another structure the student has built — never in a Python `list`, `dict`, `set` or `deque`.
- **Reason:** those built-ins hide exactly the costs the course teaches — one short call like `insert(0, x)`, `pop(0)` or `x in values` is an $O(n)$ algorithm someone else wrote. Building on a bare array makes every cost visible and yours.
- **Still allowed:** lists as function inputs and return values; lists in tests; plain local variables and tuples.
- **Examples**, any two: `Stack` on the student's `DynamicArray`; `CircularQueue` on an `Array`; `ChainingHashMap` as an `Array` of `Entry` chains; `Graph` on the student's hash map and `DynamicArrays`; `MinHeap` on a `DynamicArray`.

W2-E6 (3)

	Grows?	Holds	Hides the cost?
<code>list</code>	yes (dynamic array)	references to any objects	yes
<code>array.array</code>	yes	packed numbers of one type	yes
<code>numpy.ndarray</code>	no	packed numbers of one type	yes — whole-array operations run in C
<code>ctypes array</code>	no	C values or references	no — index and assign only

A `ctypes` array of object references is a genuine fixed-size C array that can hold any Python object. The course wraps it, adds bounds checking, and offers nothing else — so every other operation must be written, and priced, by the student.

Part C — Complexity analysis

W2-K1 — $\Theta(n^2)$. The nested loops do $n \times n$ iterations of $O(1)$ work; the following loop adds n . Sequence adds: $n^2 + n = \Theta(n^2)$.

W2-K2 — $\Theta(\log n)$. i takes the values 1, 2, 4, 8, ...; it reaches n after about $\log_2 n$ doublings. Doubling up to n is the mirror of halving down from n .

W2-K3 — $\Theta(n \log n)$. The outer loop runs n times; each time the inner loop halves j from n down to 1, about $\log_2 n$ iterations. Loops multiply. (Counting: for $n = 1,024$ the inner body runs exactly 10,240 times.)

W2-K4 — **best** $\Theta(1)$, **worst** $\Theta(n^2)$. Best: `arr[0] == arr[1]`, found on the first comparison. Worst: no duplicates, so every pair is compared: $n(n-1)/2$ comparisons.

W2-K5 — $\Theta(n^2)$. The loop body looks like one step, but `values[i:]` **copies** $n-i$ items. Total copying: $n + (n-1) + \dots + 1 = n(n+1)/2$.

W2-K6 — $\Theta(n^2)$. The middle loop is a constant 100. Total $100 \times (0 + 1 + \dots + (n-1)) = 50n(n-1)$. A large constant is still a constant. (For $n = 10$, count is 4,500.)

Part D — Proofs

W2-P1. Choose $c = 3$, $n_0 = 10$. For every $n \geq 10$, $10 \leq n$, so $2n + 10 \leq 2n + n = 3n$. Hence $2n + 10 \leq 3 \cdot n$ for all $n \geq 10$, and $2n + 10 = O(n)$. ■ (Other valid choices exist, e.g. $c = 12$, $n_0 = 1$.)

W2-P2. Suppose, for contradiction, that $n^2 = O(n)$: there are $c > 0$ and n_0 with $n^2 \leq c n$ for all $n \geq n_0$. Dividing by $n > 0$ gives $n \leq c$ for all $n \geq n_0$. But take $n = \max(n_0, \lceil c \rceil + 1)$: then $n \geq n_0$ and $n > c$ — a contradiction. So no such c exists, and n^2 is not $O(n)$. ■

W2-P3. Need $c_1, c_2 > 0$ and n_0 with $c_1 n^2 \leq 4n^2 + n \leq c_2 n^2$ for all $n \geq n_0$.

- **Lower bound:** $4n^2 + n \geq 4n^2$ for all $n \geq 0$, so $c_1 = 4$ works: $4n^2 + n = \Omega(n^2)$.
- **Upper bound:** for $n \geq 1$, $n \leq n^2$, so $4n^2 + n \leq 5n^2$; $c_2 = 5$: $4n^2 + n = O(n^2)$.
- With $n_0 = 1$ both hold, so $4n^2 + n = \Theta(n^2)$. ■

Part E — Array state

W2-S1.

(a) Shift 50, 40, 30, 20 one place right (4 moves, starting from 50), then write 15 at index 1:

```
[10, 15, 20, 30, 40, 50, _]      size 6
```

(b) Remove index 3 (the value 30): shift 40 and 50 one place left (2 moves), then clear the slot that is no longer used:

```
[10, 15, 20, 40, 50, _, _]      size 5, returned 30
```

W2-S2.

```
start          [a, b, c, d, _]
i = 1: arr[2] = arr[1]  -> [a, b, b, d, _]    c is overwritten before it moved
i = 2: arr[3] = arr[2]  -> [a, b, b, b, _]
i = 3: arr[4] = arr[3]  -> [a, b, b, b, b]
arr[1] = "X"          -> [a, X, b, b, b]
```

Copying forwards overwrites each value before it has been moved, so one value (b) is smeared across the rest and c and d are lost. **Fix:** move from the end backwards:

```
for i in range(size, index, -1):
    arr[i] = arr[i - 1]
arr[index] = value
```

W2-S3.

```
start          [1, 2, 3, 4, 5, 6]
reverse the first 2 (0..1)  [2, 1, 3, 4, 5, 6]
reverse the rest (2..5)    [2, 1, 6, 5, 4, 3]
reverse the whole (0..5)   [3, 4, 5, 6, 1, 2]
```

Each reversal is a two-pointer swap: $O(n)$ time in total, $O(1)$ extra space.

W2-S4.

(a) Value **7**, at flat index $1 \times 4 + 3 = 7$ (the values are the indices, because they were stored row by row).

(b) Row $10 // 4 = 2$, column $10 \bmod 4 = 2$: **(2, 2)**.

(c) Transposing moves (1, 3) to (3, 1) in a 4×3 matrix: flat index $3 \times 3 + 1 = \$ 10$.

Part F — Trace the code

W2-T1

```
11 13 done
```

The `else` belongs to the inner `for`: it prints `n` only when no divisor was found, i.e. `n` is prime. 10, 12, 14 and 15 hit `break`.

W2-T2

```
5 -4 0 3 ?
```

"sum" matches ["sum", *rest] with `rest = []`, so it returns 0. "add 1" has only two words, so it fails the `add` pattern (three words) and falls to `_`.

W2-T3

```
A B D E 5
A C E 0
```

`finally` always runs, **before** the returned value reaches `print`. With 0, the division raises, so `B` is skipped and `except` prints `C`; `else` runs only when the `try` did not raise.

W2-T4

```
[1, 2] 2 20
```

`values.append(n)` mutates the caller's list. `n = n * 10` rebinds a local name — `k` is untouched. `values = [0]` rebinds the local name too — `data` keeps `[1, 2]`.

Part G — Find and fix the bug

W2-B1. `range(2, int(n ** 0.5))` stops **before** the square root, so a perfect square of a prime is never tested against its root: `is_prime(9)`, `is_prime(25)` and `is_prime(49)` return `True`. **Fix:** `range(2, int(n ** 0.5) + 1)` — or, avoiding floats, `d = 2; while d * d <= n:`.

W2-B2. The `else: return -1` is inside the loop, so the function gives up after checking **only** `arr[0]`. `find([4, 7], 7)` returns `-1`. **Fix:** move `return -1` after the loop, and delete the `else`.

W2-B3. The loop runs one step too far: for `i = size - 1` it reads `arr[size]`, one past the used part — an `IndexError` when the array is full (`size == len(arr)`), and a silent read of a free slot otherwise. The slot that is no longer used is also never cleared. **Fix:**

```
for i in range(index, size - 1):
    arr[i] = arr[i + 1]
arr[size - 1] = None
```

Part H — Write the code

W2-C1

```
def count_occurrences(arr, target, size=None):
    n = len(arr) if size is None else size
    count = 0
    for i in range(n):
        if arr[i] == target:
            count += 1
    return count
```

W2-C2 — a *read* index and a *write* index.

```
def remove_all(arr, size, target):
    write = 0                                # next slot to keep a value in
    for read in range(size):
        if arr[read] != target:
            arr[write] = arr[read]
            write += 1
    for i in range(write, size):              # clear the freed tail
        arr[i] = None
    return write
```

Each element is read once and written at most once: $O(n)$, $O(1)$ extra. Calling `remove_at` for each occurrence shifts up to n elements **per removal**; with k occurrences that is $O(kn)$, which is $O(n^2)$ when k is proportional to n .

W2-C3 — two indices moving towards each other.

```
def two_sum_sorted(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        total = arr[lo] + arr[hi]
        if total == target:
            return (lo, hi)
        if total < target:
            lo += 1                                # need a bigger sum
        else:
            hi -= 1                                # need a smaller sum
    return None
```

Why it is correct: if $\text{arr}[\text{lo}] + \text{arr}[\text{hi}]$ is too small, then $\text{arr}[\text{lo}]$ plus *any* element at or before hi is also too small (the array is sorted), so $\text{arr}[\text{lo}]$ can be in no solution — discard it. Symmetrically for too big. Each step discards one element, so at most $n - 1$ steps: $O(n)$.

W2-C4

```
def prefix_sums(arr):  
    result = Array(len(arr))  
    running = 0  
    for i in range(len(arr)):  
        running += arr[i]  
        result[i] = running  
    return result
```

Then $\text{sum}(\text{arr}[i..j]) = p[j] - p[i - 1]$ (or $p[j]$ when $i = 0$): one subtraction, $O(1)$, after one $O(n)$ pass. Paying once up front to answer many questions cheaply is a trade you will see again and again.